Des tentatives de connexions ont été répertoriées sur plusieurs comptes. Si votre compte a été bloqué (locked), activez l'authentification double facteur sur votre compte. Vous pouvez utiliser votre téléphone pour valider votre connexion à GitLab.

```
subtitle: Les patrons de conception en *Java*
```

Download

Les patrons de conception en *Java*

Les **patrons de conception** sont des *modèles* de conception, fournissant des *solutions* à des problèmes de conception fréquents, en utilisant des *structures génériques* pouvant être *étendues* ou *adaptées* à des cas d'utilisation précis, et facilitant la *réutilisation* du code existant. Pour reprendre les mots de Christopher Alexander, *« chaque patron décrit un problème qui apparaît encore et encore dans notre environnement, et décrit la base de la solution à ce problème, de manière telle que vous pouvez utiliser cette solution des millions de fois sans jamais le faire de la même manière »*. Cependant, un patron n'est pas juste une « recette » qui aide à structurer le code : c'est aussi une façon de communiquer à d'autres développeurs le problème qu'il résout.

Le but de ce cours est de vous présenter différents patrons de conception classiques de la programmation orientée objet en *Java*, en particulier ceux décrits dans le livre *Design Patterns: Elements of Reusable Object-Oriented Software* (Erich Gamma, Richard Helm, Ralph Johnsons et John Vlissides, a.k.a. le *Gang of Four*). À l'issue de ce cours, vous devriez être capable de :

- **Identifier** quel patron de conception permet de résoudre un problème donné.
- **Implémenter** ce patron de conception.
- **Adapter** ce patron de conception suivant vos besoins.

Table des matières

- Les patrons de conception en Java
  - Table des matières
  - Conseil de lecture
  - Patrons de création
    - La fabrique abstraite (abstract factory)
    - Le constructeur (builder)
    - Le singleton
  - Patrons structurels
    - L'adaptateur (adapter)
    - Le composite
    - Le décorateur (decorator)
    - Le proxy
  - Patrons comportementaux
    - L'itérateur (iterator)
    - L'état (state)
    - La stratégie (strategy)
    - Le patron de méthode (template method)
    - Le visiteur (visitor)
  - Application des patrons de conception : retour sur les fichiers
    - Un proxy pour limiter l'accès aux informations d'un fichier
    - Une stratégie pour les algorithmes de chiffrement.
    - Un décorateur pour chiffrer les fichiers
    - Un adaptateur pour utiliser la classe java.io.File pré-existante
    - Une fabrique abstraite pour créer les fichiers
    - Un état pour verrouiller les fichiers
    - Un singleton pour des états uniques
    - Le patron de méthode pour le stockage dans le cloud
- Le composite pour les répertoires
- L'itérateur pour parcourir les fichier d'un répertoire
- Le visiteur pour explorer la hiérarchie des fichiers

Conseil de lecture

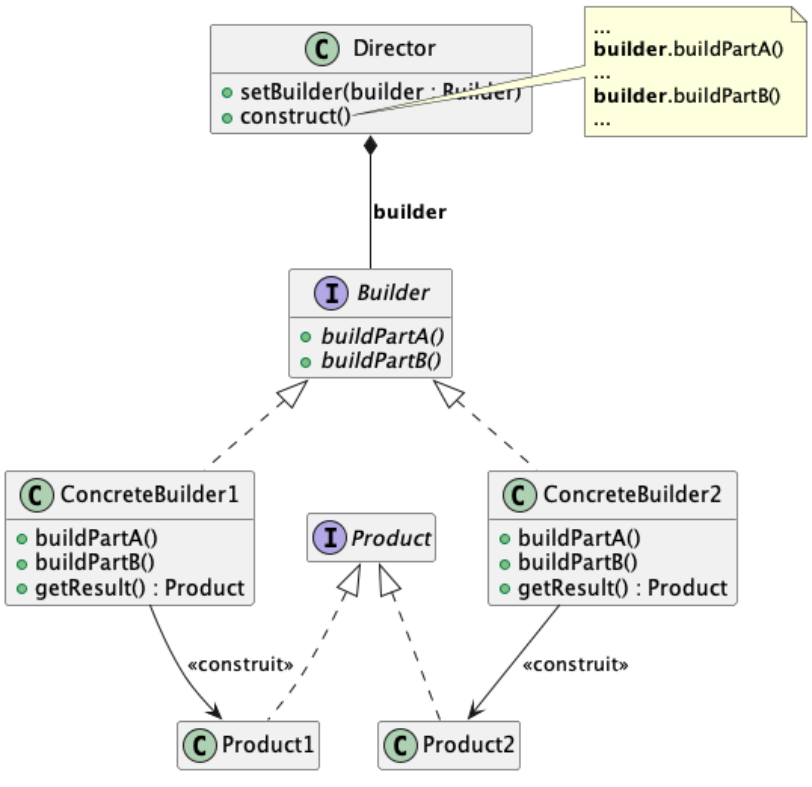
Ce cours présente dans un premier temps l'ensemble des patrons de conception étudiés avant d'en donner des exemples d'utilisation sur un cas concret, afin de pouvoir également présenter leurs interactions. N'hésitez pas à alterner entre la description d'un patron et son exemple associé (par exemple, en utilisant la table des matières ci-dessus) afin de vous aider à mieux comprendre son fonctionnement.

Patrons de création

Dans cette première section sont présentés les patrons de conception permettant la *création* d'objets. Ces patrons permettent de cacher le type concret des objets créés, et comment ils sont construits. Ils permettent beaucoup plus de flexibilité que les constructeurs concernant *qui* construit *quoi*, *comment* et *quand*.

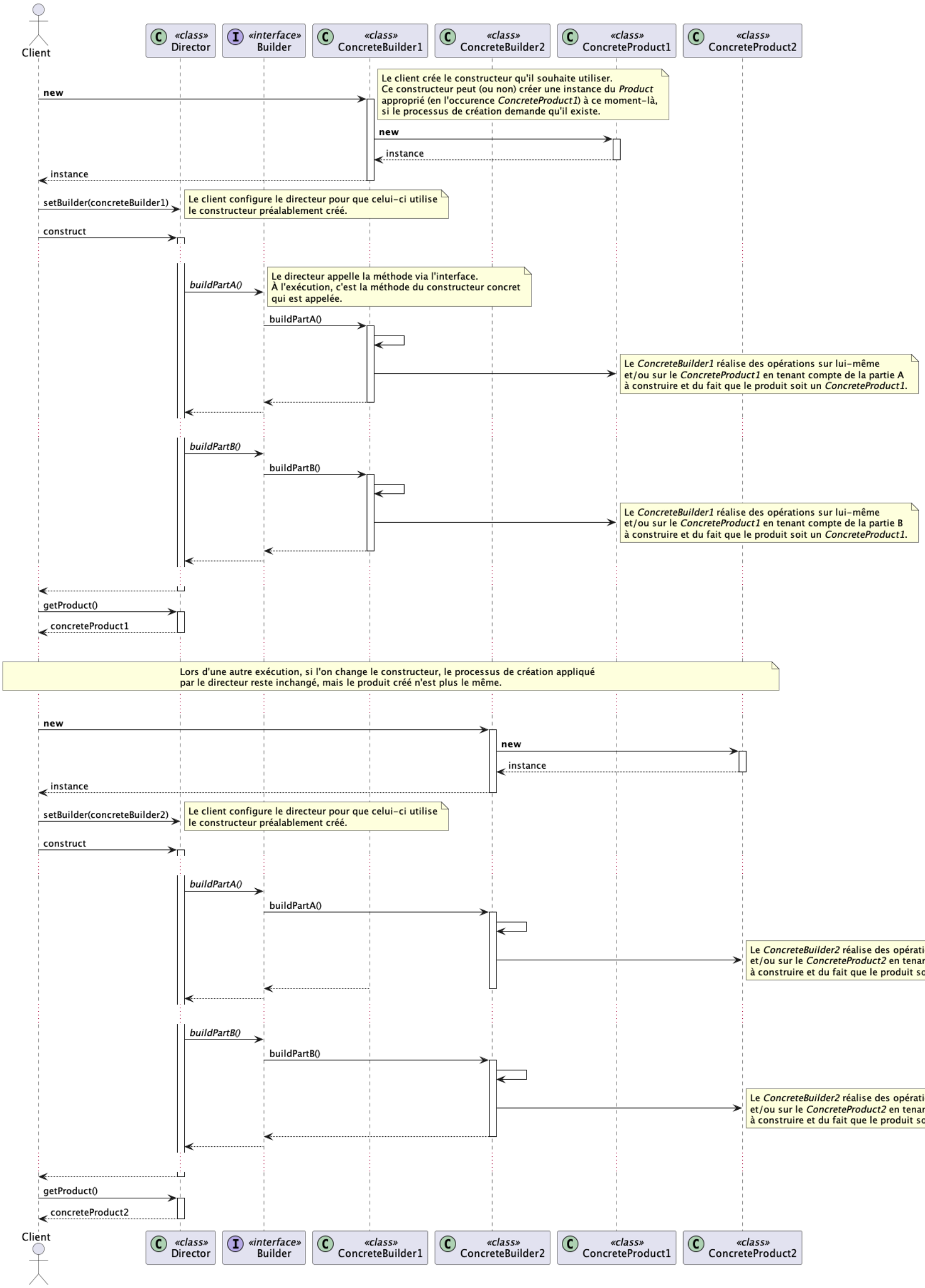
La fabrique abstraite (*abstract factory*)

La *fabrique abstraite* permet la création d'objets dont le type concret n'est pas connu à l'avance.

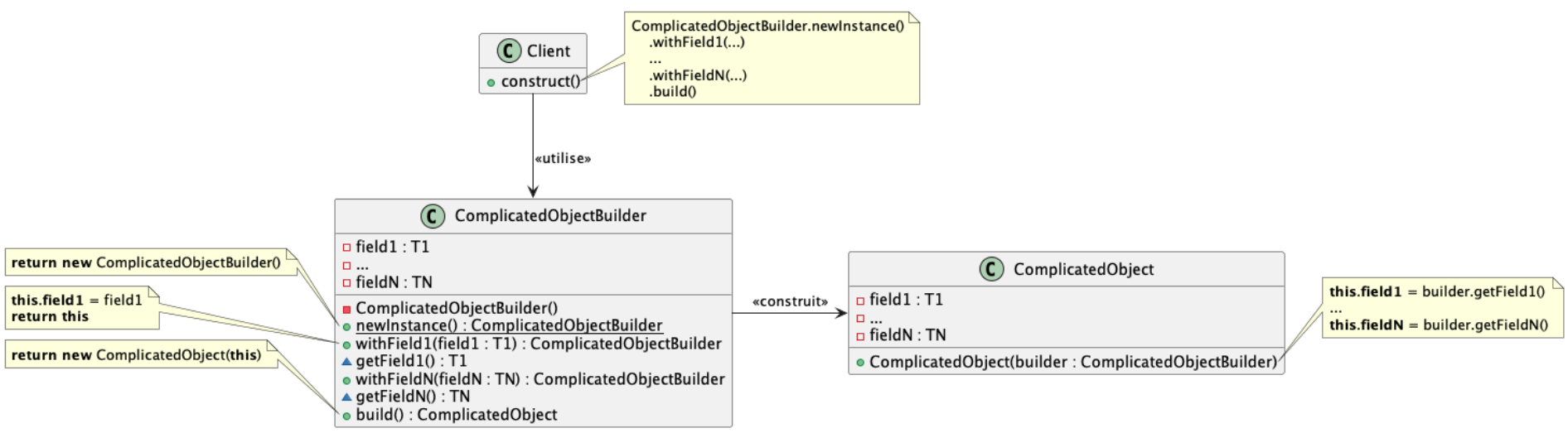


En changeant l'implémentation du `Builder` utilisée par le `Director`, il est possible de créer des objets partageant une structure relativement proche, tout en utilisant des types différents (dont l'instanciation est laissée à l'implémentation du `Builder` utilisée).

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception constructeur.

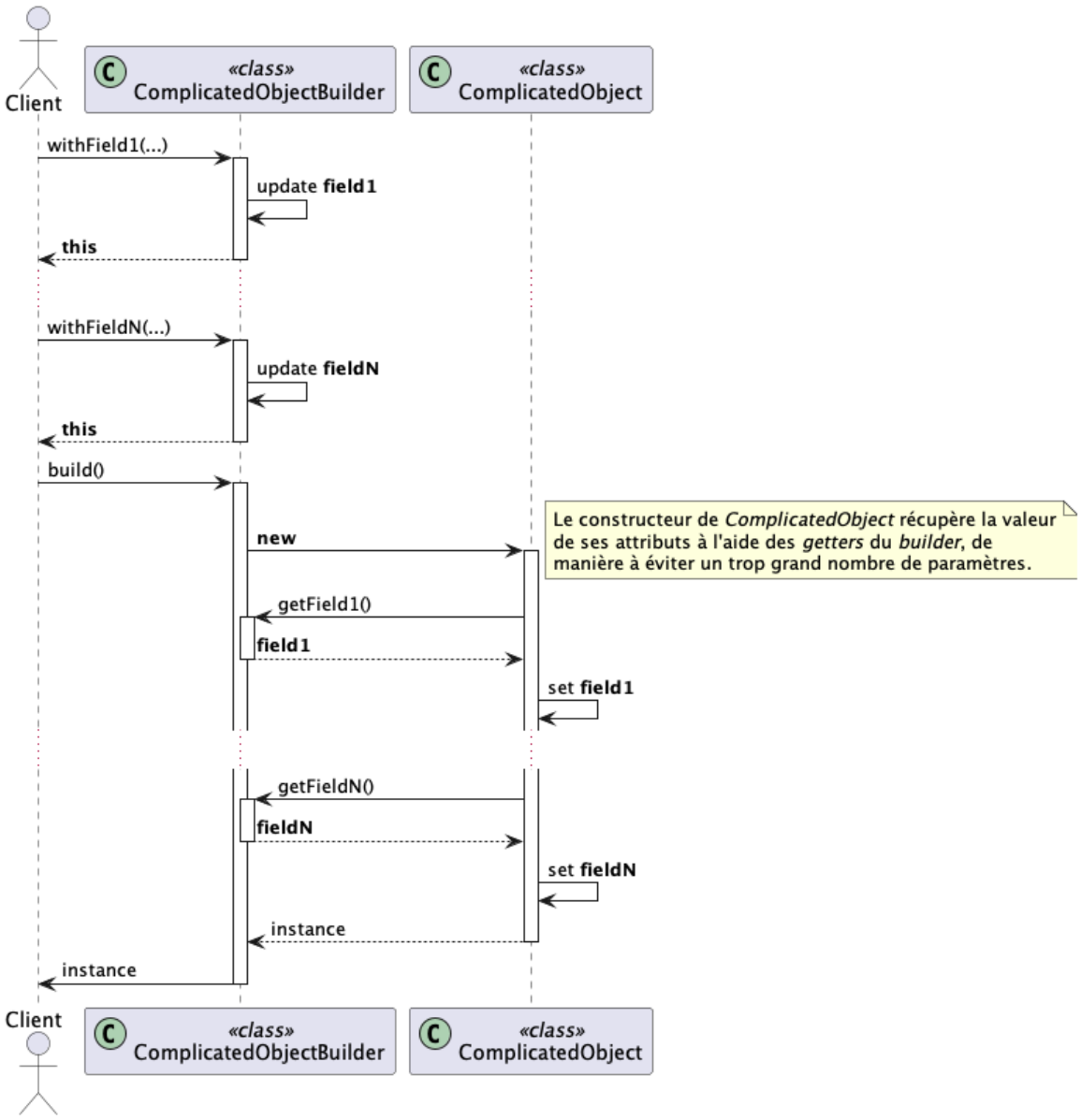


Une variante du patron constructeur vise à associer un à un les attributs de l'objet à construire au travers du constructeur.



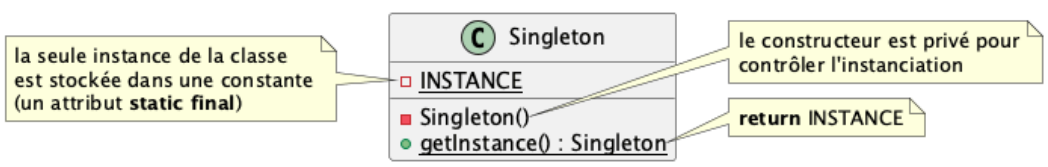
Dans ce cas, l'objet est construit progressivement à l'aide du constructeur, et lorsque l'ensemble des attributs nécessaires sont initialisés, l'objet final peut être créé à partir du constructeur lui-même. Ce processus de construction permet ainsi d'éviter d'avoir trop de paramètres dans le constructeur d'une classe, ce qui peut nuire à la lisibilité du code et être **source d'erreurs** (notamment, en cas d'inversion de paramètres).

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants de cette variante du patron de conception constructeur.



Le singleton

Le *singleton* garantit qu'une classe ne possède qu'une seule instance.



L'utilisation d'un singleton permet plus de flexibilité que l'utilisation de méthodes statiques. En particulier, il est toujours possible de revenir en arrière et d'autoriser la création de plus d'une instance de la classe, ou encore de retourner une instance d'une sous-classe plutôt qu'une instance de la classe elle-même. De plus, un singleton peut implémenter une interface ou hériter d'une classe, ce qui n'est pas compatible avec l'utilisation de méthodes statiques.

Attention cependant à ce que le singleton ne devienne pas un prétexte pour définir des variables globales !

Notez qu'en *Java*, il est tout à fait possible d'utiliser une énumération plutôt qu'un singleton. Procéder de cette manière fournit des garanties plus fortes sur la création d'instances, comme les restrictions sont directement intégrées au langage.

Patrons structurels

Dans cette section sont présentés les patrons exploitant la *composition* des objets et classes en « superstructures ». Plus précisément, nous présentons ici la composition *dynamique* permise par les patrons d'objets, plutôt que la composition statique imposée par les patrons de classes (fondés sur l'héritage).

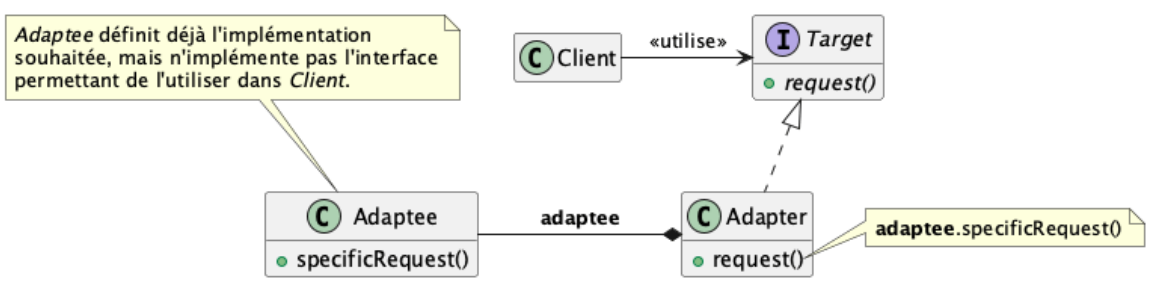
En effet, l'héritage est souvent moins flexible que la composition, notamment car les sous-classes *dépendent* de l'implémentation de la classe parente : si elle est modifiée, les sous-classes sont impactées, et peuvent même ne plus fonctionner comme prévu. De plus, redéfinir certaines méthodes peut être **source d'erreurs** (par exemple, en cas d'oubli d'opérations requises), voire être **impossible** (par exemple, si des attributs ou des méthodes sont privés). Enfin, il n'est possible d'hériter que d'une seule classe en *Java*, et tous ses attributs sont alors hérités, même si ils ne sont pas nécessaires.

De manière générale, vous devriez toujours appliquer la règle suivante : avant d'écrire une classe `B` qui étend une classe `A`, demandez-vous toujours si le fait que `B` **soit** un `A` a vraiment du sens, et n'hésitez pas à déclarer classes et méthodes comme `final` pour empêcher l'héritage ou la redéfinition.

Pour toutes ces raisons, nous omettons volontairement les variantes des patrons présentés ici utilisant l'héritage. Cependant, notez qu'elles existent, et qu'il peut donc arriver que vous les rencontriez dans des bases de code existantes.

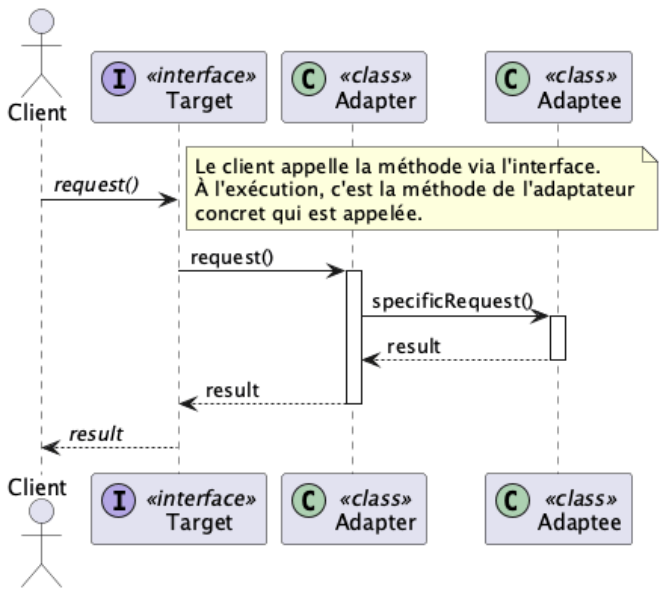
L'adaptateur (*adapter*)

L'*adaptateur* convertit l'interface d'une classe en une autre, pour permettre à des classes initialement incompatibles de communiquer ensemble.



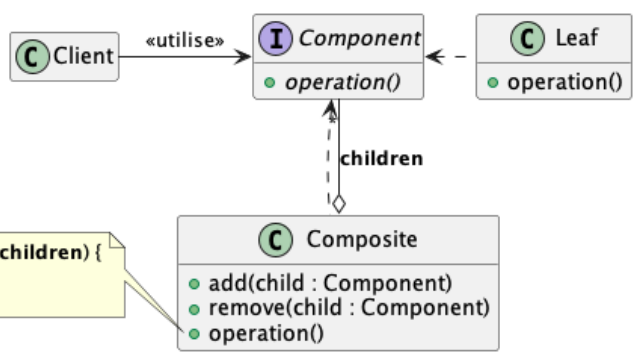
L'adapatateur permet ainsi d'exploiter le code existant fourni par la classe adaptée sans avoir besoin de le dupliquer, tout en permettant de remplacer cette classe par n'importe laquelle de ces sous-classes.

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception adaptateur.



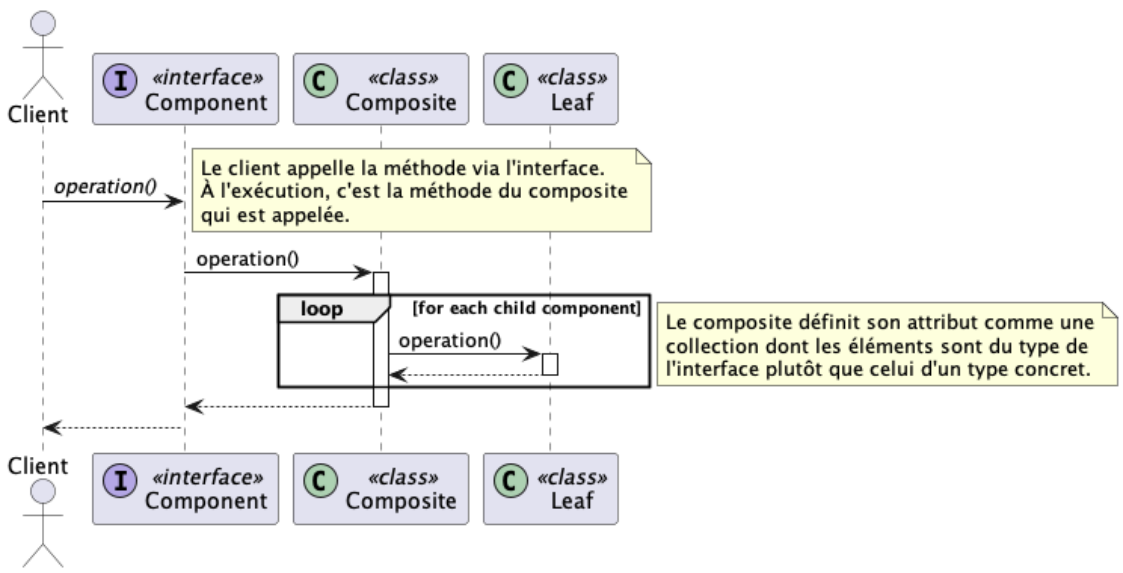
Le composite

Le *composite* permet de traiter uniformément des objets individuels ou des collections d'objets à travers une structure d'arbre.



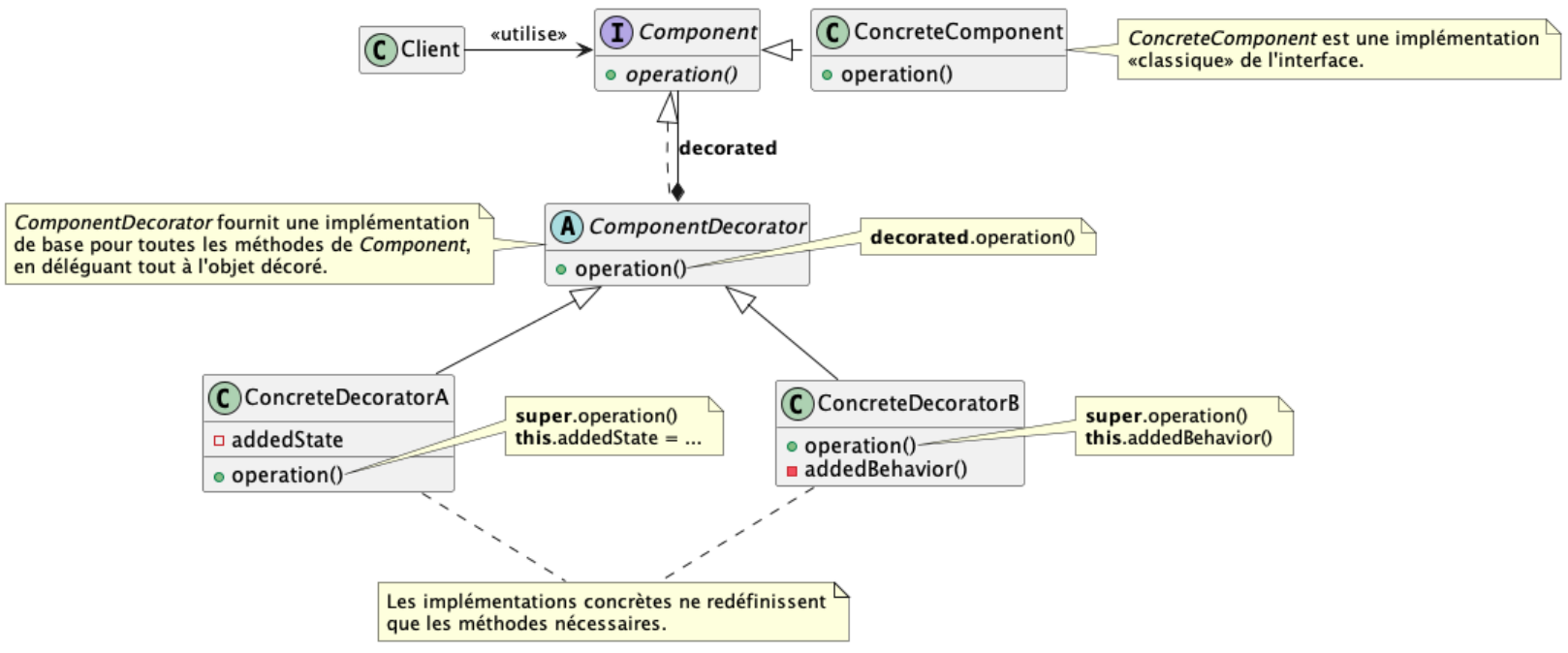
En utilisant un composite, il devient possible de traiter une collection d'objets implémentant `Component` partout où une seule instance est attendue dans le code. Le `Client` n'a donc jamais à se soucier de la quantité d'objets qu'il doit traiter. Il est aussi plus simple d'ajouter des nouveaux composants : le code du client fonctionne avec n'importe quelle structure dont la racine est une implémentation de `Component`.

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception composite.



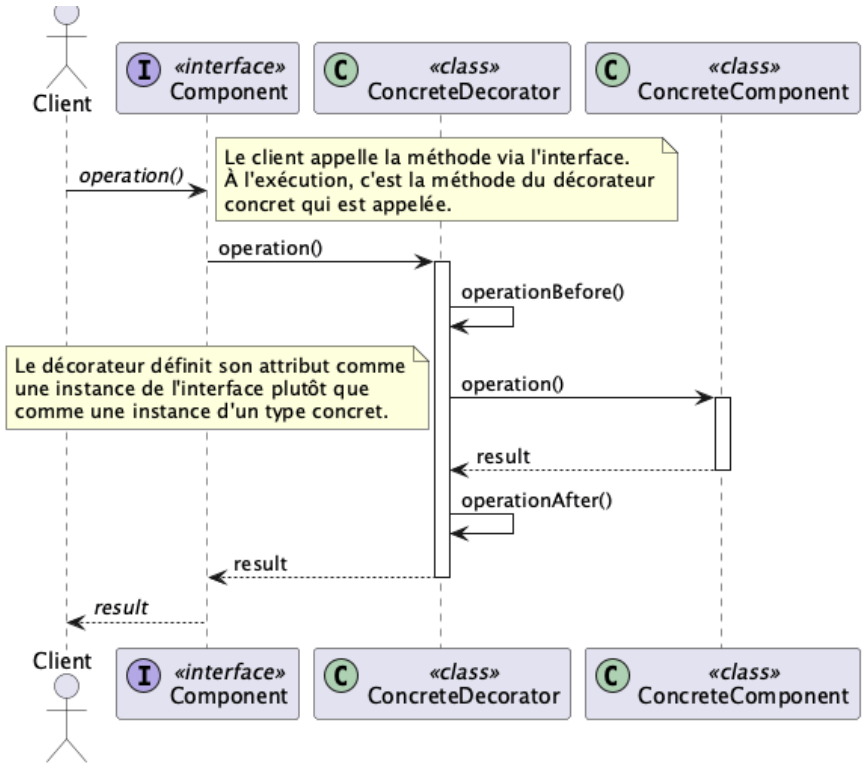
Le décorateur (*decorator*)

Le *décorateur* permet d'ajouter dynamiquement des fonctionnalités à un objet existant.



Le décorateur offre plus de flexibilité que l'héritage : il permet en effet d'ajouter (ou retirer) des fonctionnalités à un objet pendant l'exécution de l'application, et ce indépendamment de son type concret. En particulier, le décorateur fonctionne avec n'importe quelle implémentation de l'interface `Component`, ce qui évite de dupliquer les fonctionnalités qu'il ajoute pour chacune de ces implémentations.

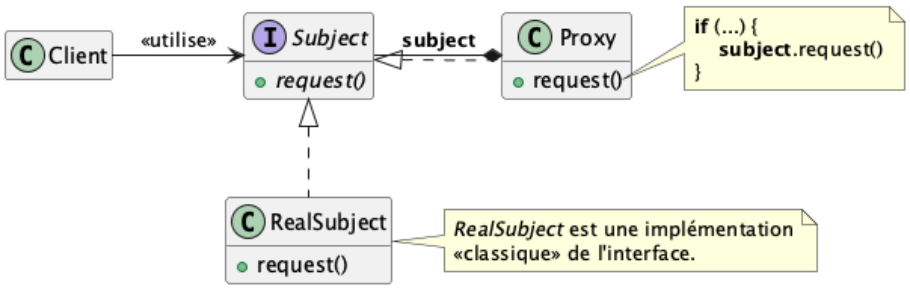
Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception décorateur.


Client
«interface»
Component
«class»
ConcreteDecorator
«class»
ConcreteComponent
operation()
Le client appelle la méthode via l'interface.
À l'exécution, c'est la méthode du décorateur
concret qui est appelée.
operation()
operationBefore()
Le décorateur définit son attribut comme
une instance de l'interface plutôt que
comme une instance d'un type concret.
operation()
result
operationAfter()
result
result
Client
«interface»
Component
«class»
ConcreteDecorator
«class»
ConcreteComponent

Le proxy

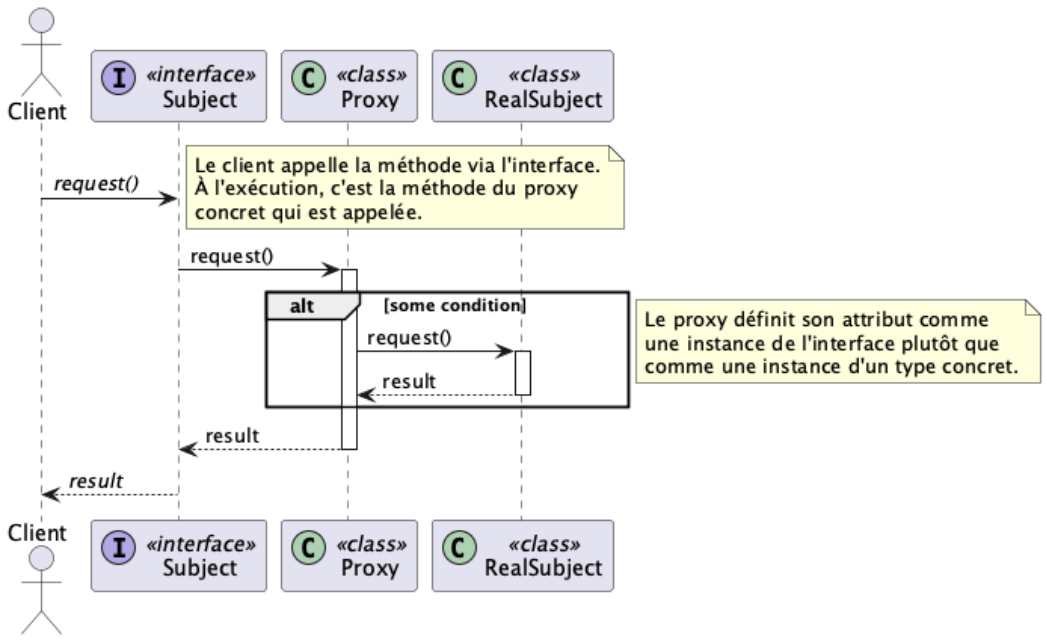
Le proxy

Le *proxy* contrôle les accès aux méthodes d'un objet existant.



Le proxy offre plus de flexibilité que l'héritage : il permet en effet de réaliser des vérifications en amont des appels de méthodes sur un objet existant pendant l'exécution de l'application, et ce indépendamment de son type concret et de manière transparente pour le `Client`. En particulier, le proxy fonctionne avec n'importe quelle implémentation de l'interface `Subject`, ce qui évite de dupliquer les fonctionnalités qu'il ajoute pour chacune de ces implémentations.

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception proxy.

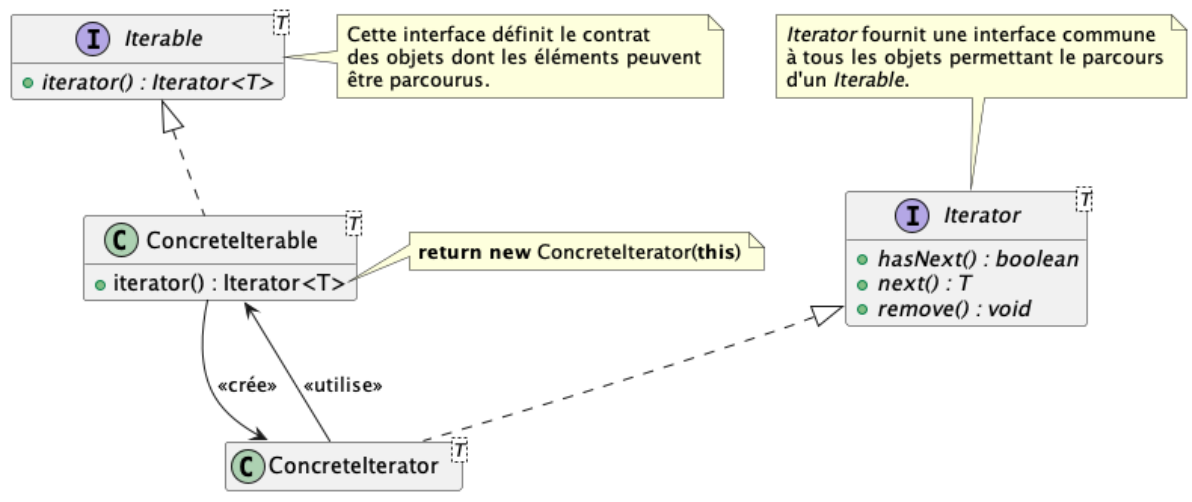


Patrons comportementaux

Dans cette section sont présentés les patrons de conception *comportementaux*, qui permettent de *gérer des algorithmes*, des *responsabilités* et de *contrôler les communications* entre les objets.

L'itérateur (*iterator*)

L'*itérateur*, permet de parcourir les éléments d'une collection d'objets.



En *Java*, il existe une pré-implémentation de ce patron de conception, que vous devez respecter pour bénéficier des avantages proposés par le langage. Par exemple, l'interface `Collection<E>` hérite de l'interface `Iterable<E>`. Il est donc possible de parcourir une collection de la manière suivante :

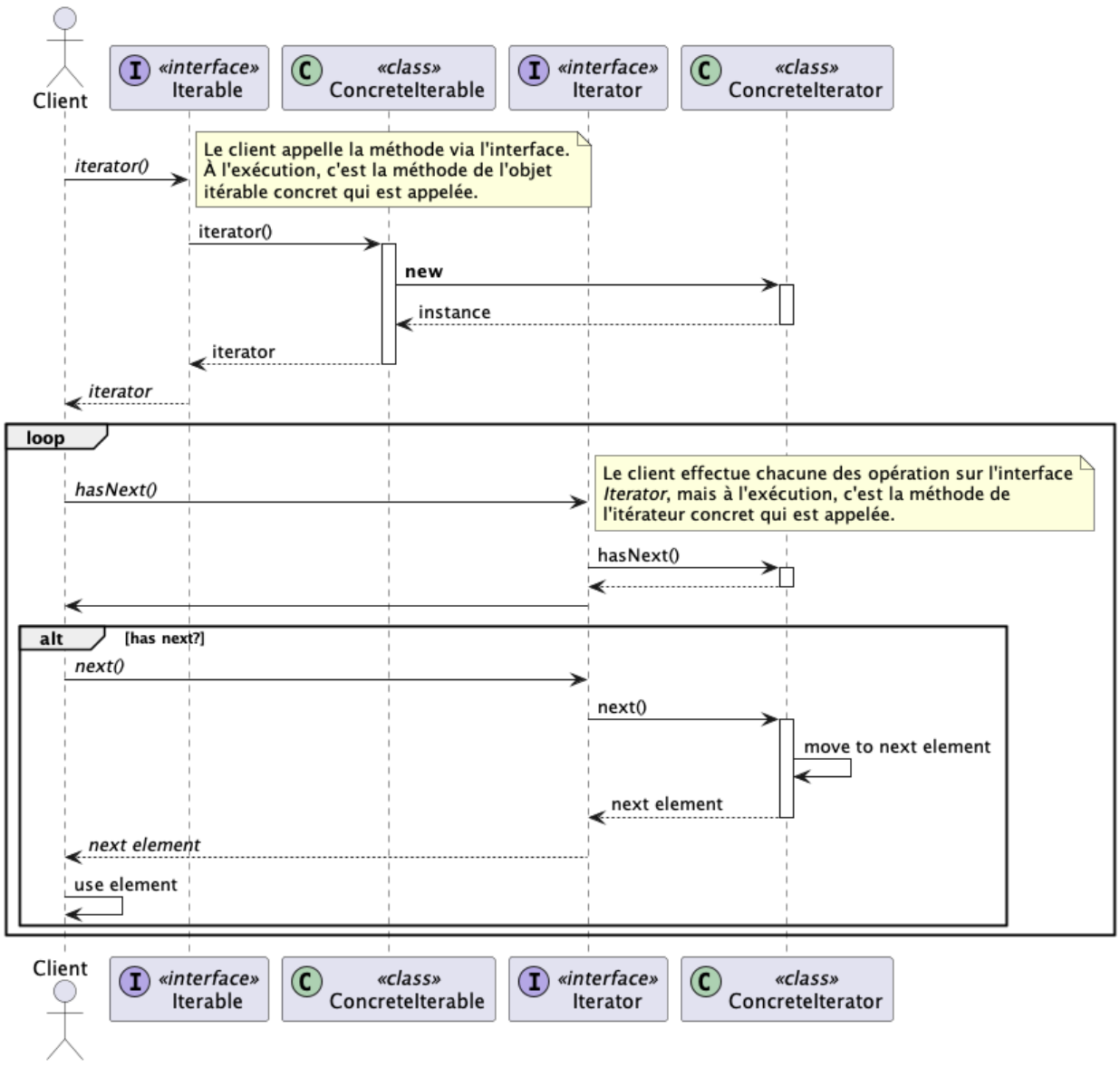
```
for (Iterator<MonElement> it = maCollection.iterator(); it.hasNext();) {
  MonElement element = it.next();
  // Faire quelque chose avec l'élément courant...
}
```

En pratique, le `for-each` est souvent préféré pour sa syntaxe plus claire et concise, comme dans l'exemple ci-dessous :

```
for (MonElement element : maCollection) {
  // Faire quelque chose avec l'élément courant...
}
```

Les deux exemples ci-dessus sont strictement équivalents. En fait, il s'agit de *sucre syntaxique* : le compilateur se charge de traduire la seconde forme dans la première.

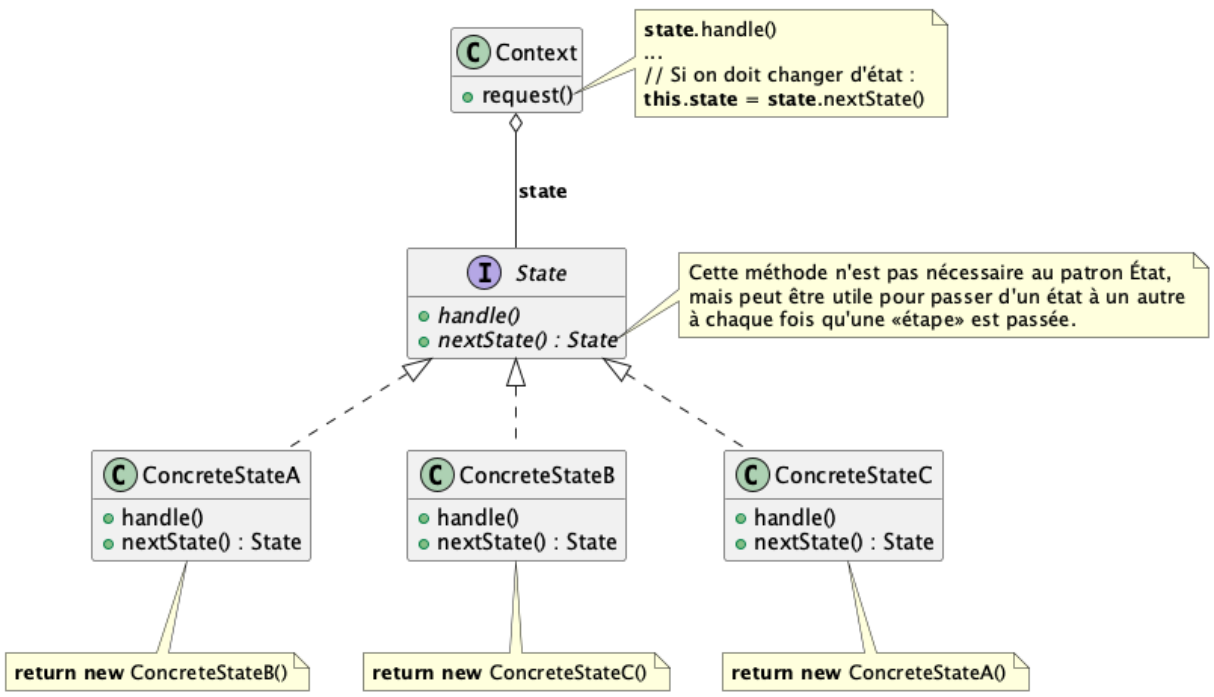
Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception itérateur.



Il est ici important de noter qu'un `Iterable` peut être parcouru un nombre de fois arbitraire, tandis qu'un `Iterator` est à usage unique.

L'état (*state*)

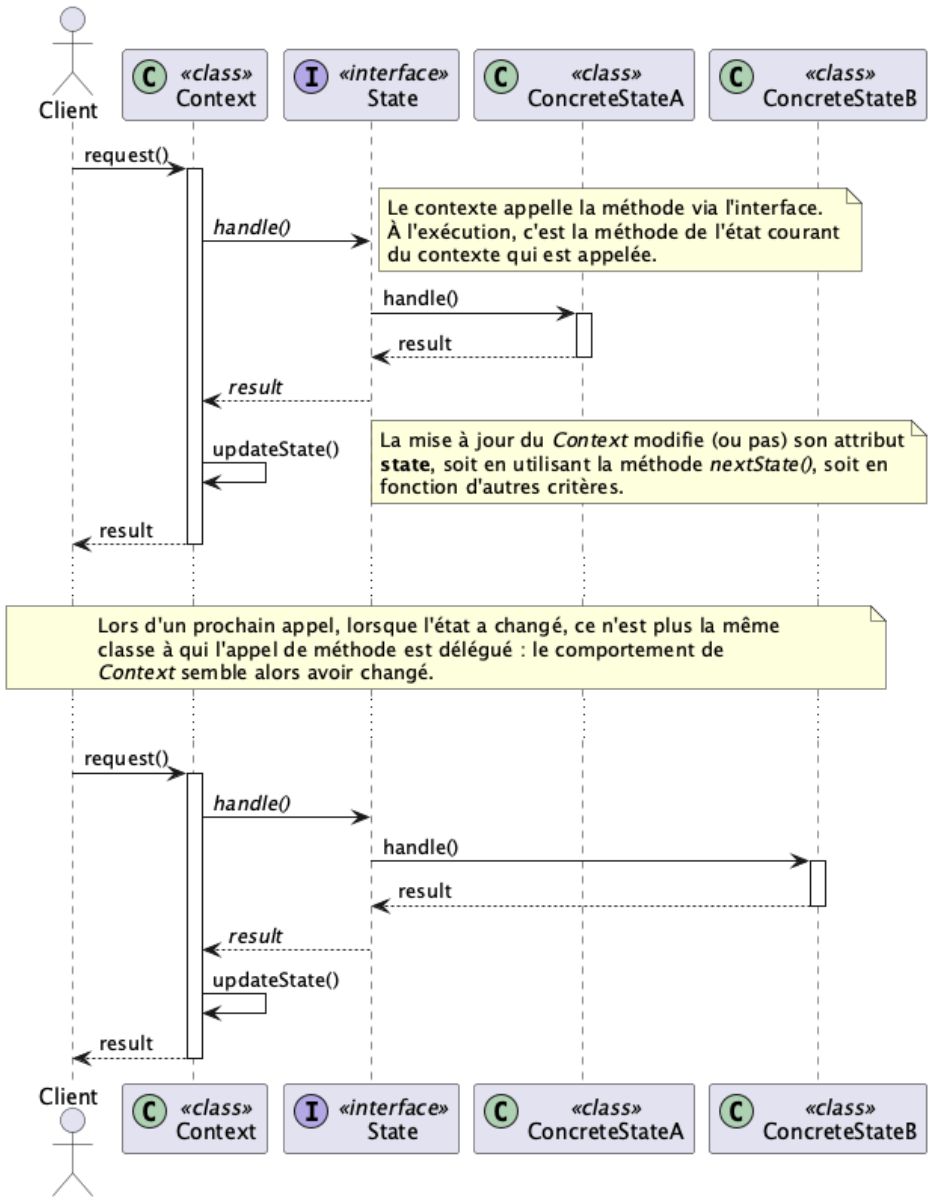
L'*état* modifie le comportement d'un objet lorsque son état interne change, comme si l'objet changeait de classe.



Le patron état permet de séparer dans différentes classes le code propre à chacun des états, ce qui rend plus explicite la différence entre ces états et le passage d'un état à un autre. Il est aussi plus simple de faire évoluer les états et en particulier d'en ajouter : il suffit essentiellement d'ajouter de nouvelles classes implémentant `State`.

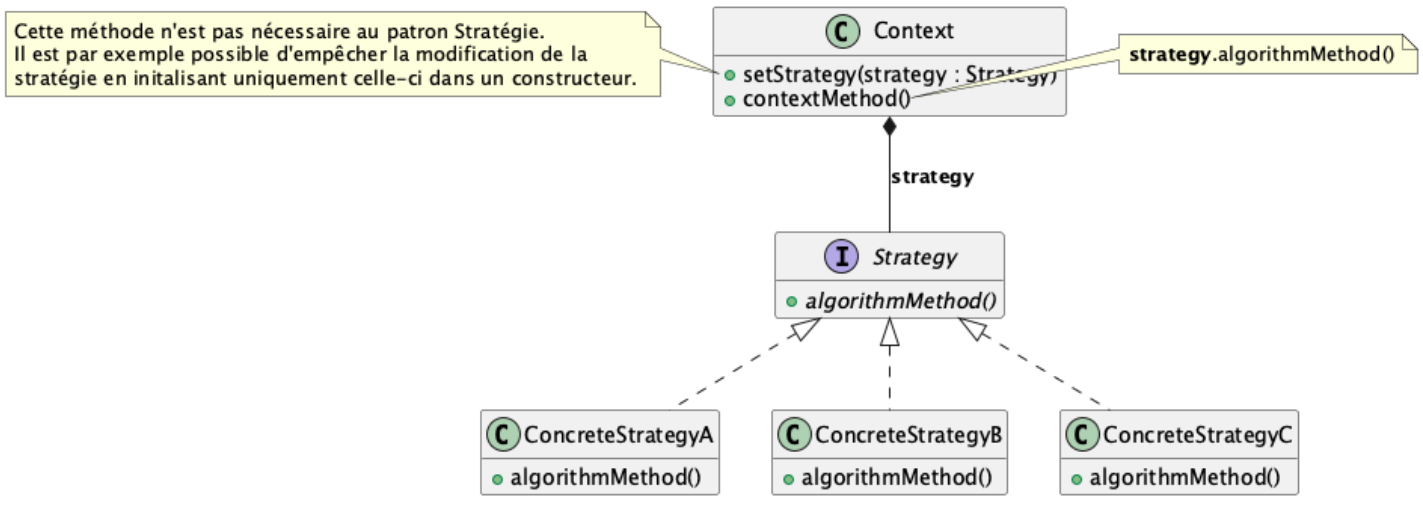
Ce patron permet également d'éviter de nombreuses instructions conditionnelles : plutôt que de devoir tester différentes conditions pour déterminer l'état actuel d'un objet et exécuter le code approprié, il suffit d'exploiter le polymorphisme d'un appel de méthode sur un objet implémentant `State`.

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception état.



La stratégie (*strategy*)

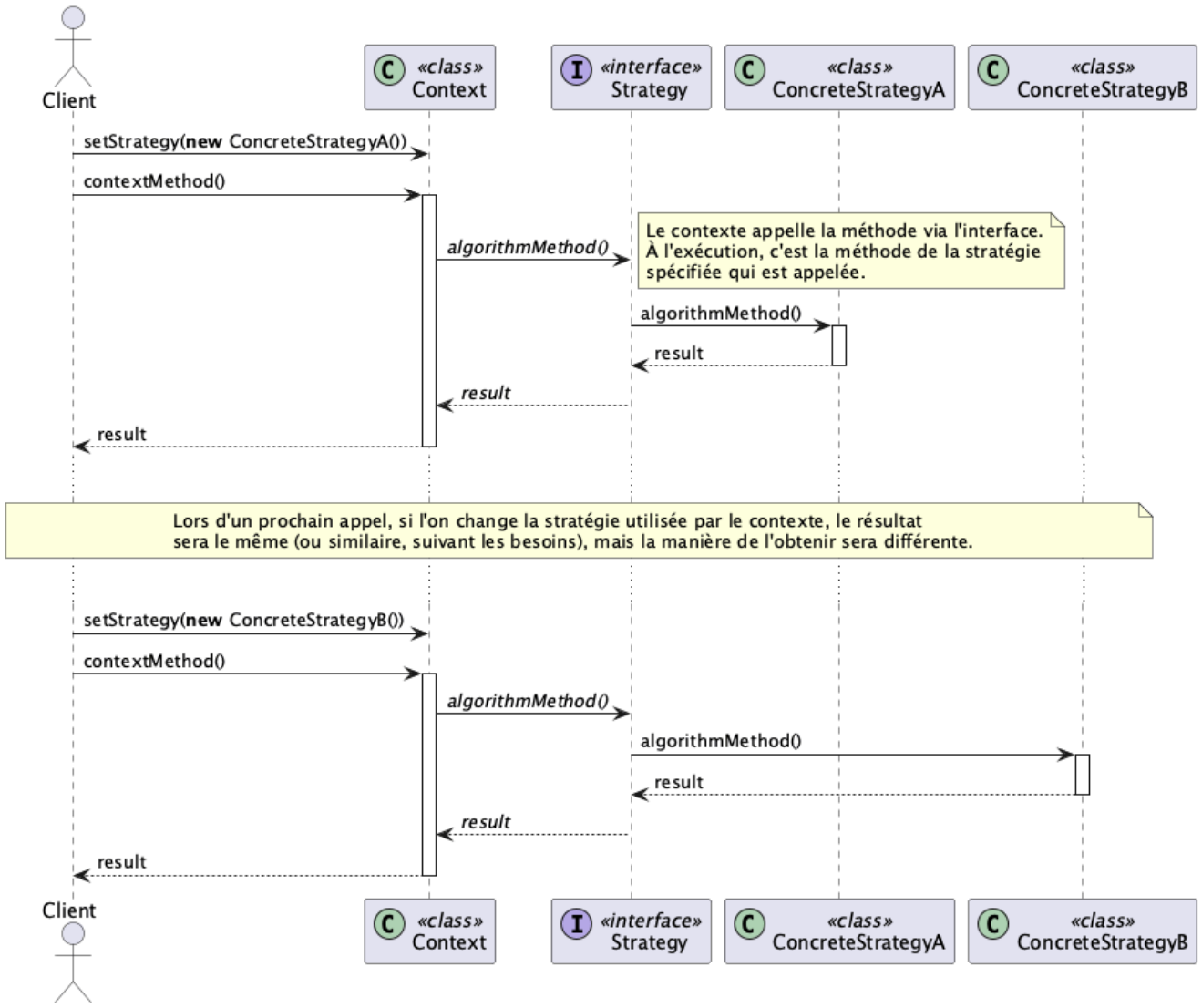
La *stratégie* permet de rendre interchangeables des algorithmes ayant des rôles similaires.



Le patron stratégie permet une grande flexibilité dans l'utilisation de différents algorithmes d'une même famille. Il permet notamment d'éviter d'utiliser l'héritage pour spécifier certains comportements, ce qui pourrait empêcher de les réutiliser dans d'autres classes ou qui nécessiterait de les dupliquer. Il est aussi plus simple d'ajouter de nouvelles stratégies : il suffit essentiellement d'ajouter de nouvelles classes implémentant `Strategy`.

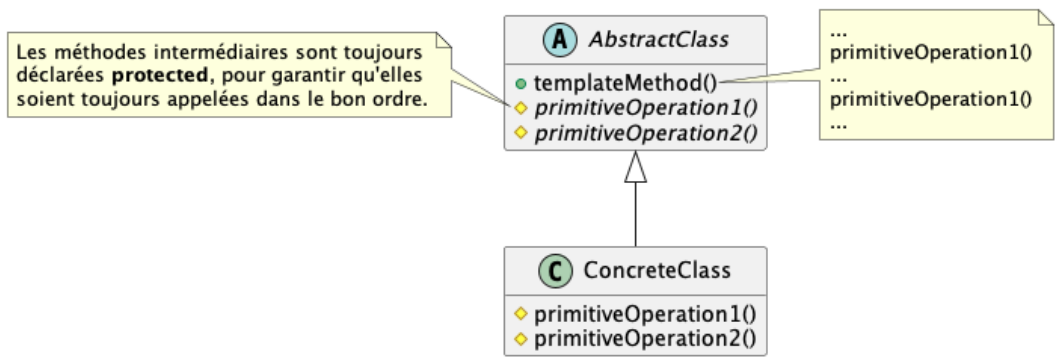
Ce patron permet également d'éviter de nombreuses instructions conditionnelles : plutôt que de devoir tester différentes conditions pour déterminer une configuration à adopter, il suffit d'exploiter le polymorphisme d'un appel de méthode sur un objet implémentant `Strategy` judicieusement choisi à la création du `Context`.

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception stratégie.



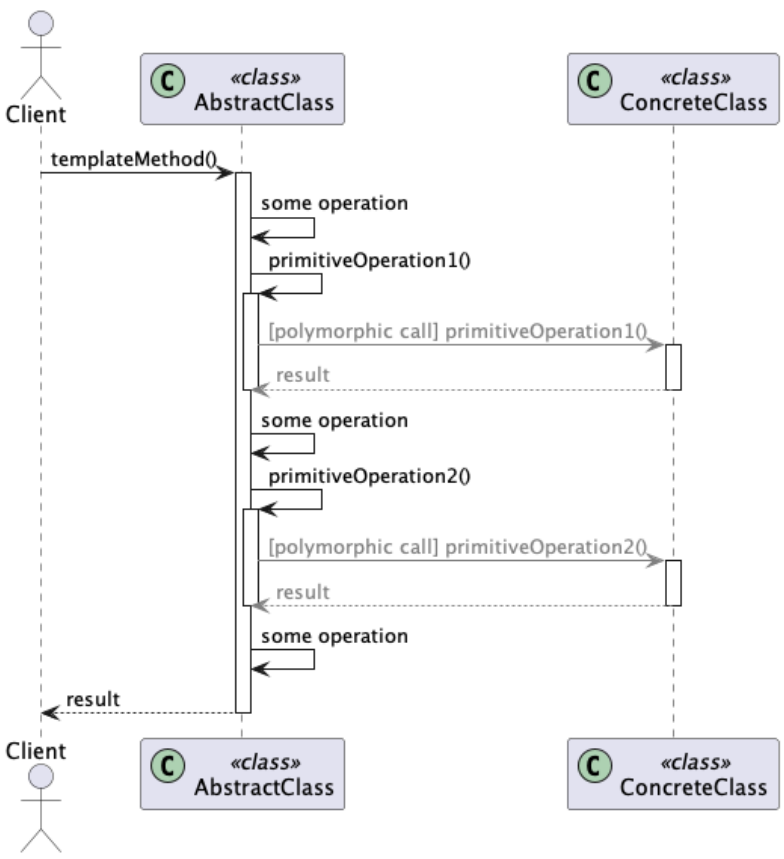
Le *patron de méthode* (*template method*)

Le *patron de méthode* définit un squelette d'algorithme dans lequel les étapes intermédiaires sont implémentées par les sous-classes.



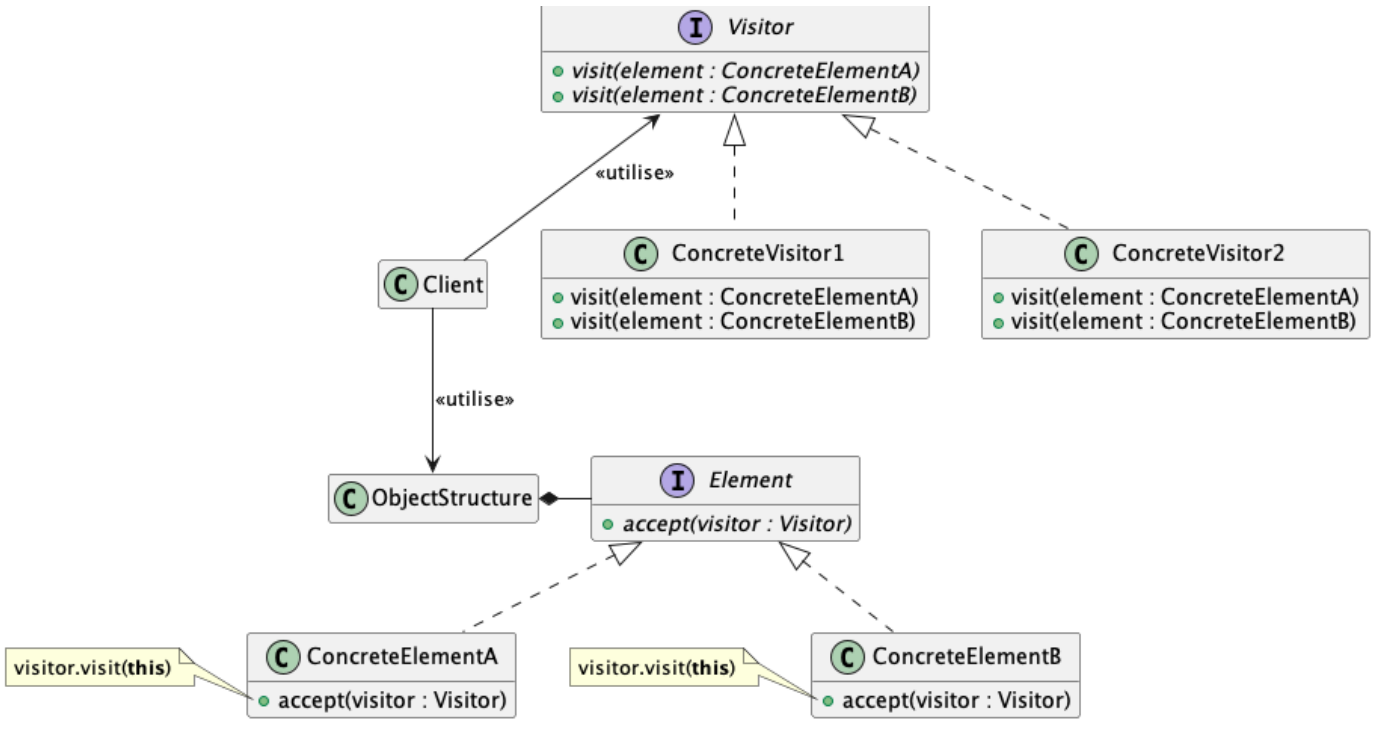
Le patron de méthode permet ainsi de factoriser le squelette de l'algorithme pour éviter la duplication de code. Il permet également d'assurer l'ordre dans lequel les étapes de l'algorithme seront exécutées, tout en laissant la liberté aux sous-classes de modifier le comportement de ces étapes, mais également d'empêcher la redéfinition de certaines d'entre elles (par exemple, en rendant certaines méthodes `private` ou `final`).

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de méthode.



Le *visiteur* (*visitor*)

Le visiteur représente une opération à réaliser sur des éléments d'une structure d'objets.


Visitor
visit(element : ConcreteElementA)
visit(element : ConcreteElementB)
«utilise»
ConcreteVisitor1
visit(element : ConcreteElementA)
visit(element : ConcreteElementB)
ConcreteVisitor2
visit(element : ConcreteElementA)
visit(element : ConcreteElementB)
Client
«utilise»
ObjectStructure
Element
accept(visitor : Visitor)
visitor.visit(this)
ConcreteElementA
accept(visitor : Visitor)
visitor.visit(this)
ConcreteElementB
accept(visitor : Visitor)

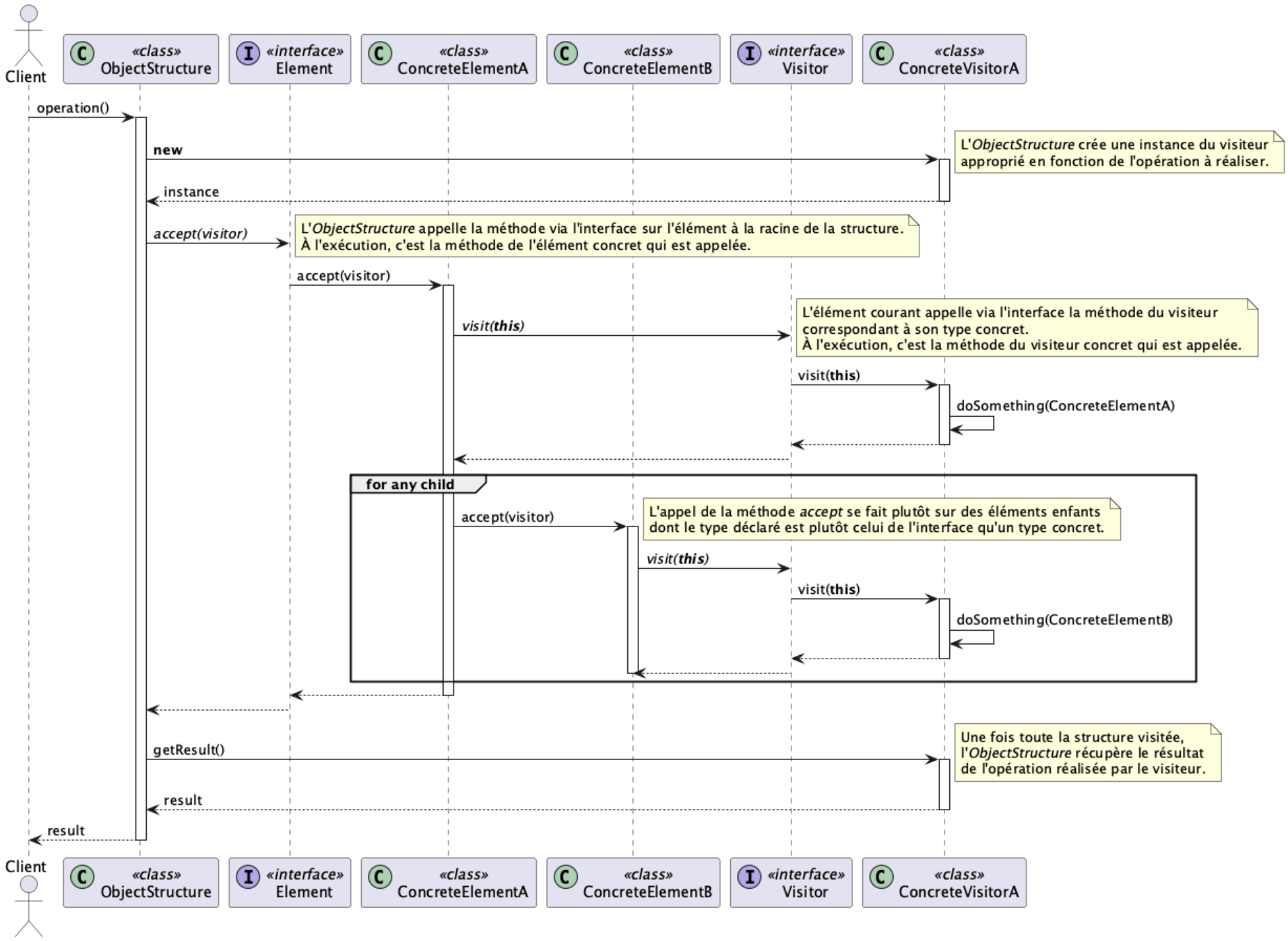
Le visiteur permet de séparer la représentation objet de la structure explorée et les opérations à réaliser dessus. Il devient alors facile d'ajouter de nouvelles opérations : il suffit de définir un

Le visiteur permet de séparer la représentation objet de la structure explorée et les opérations à réaliser dessus. Il devient alors facile d'ajouter de nouvelles opérations : il suffit de définir un

nouveau visiteur. Chaque visiteur ne considère que les informations dont il a besoin, ce qui rend chaque visiteur facilement maintenable individuellement.

Attention cependant : l'ajout de nouvelles classes visitées (*i.e.*, de nouvelles implémentations de `Element`) est plus complexe. Il requiert l'ajout de méthodes dans l'interface `Visitor` (une par nouvelle classe), ce qui a donc des conséquences dans chacune des classes concrètes qui l'implémentent, y compris celles qui pourraient avoir été définies à l'extérieur de l'application (par exemple, dans le cas d'une bibliothèque). Il est donc préférable d'utiliser le visiteur sur une hiérarchie d'objets qui n'évoluera pas (ou peu) sur le long terme.

Le diagramme de séquence ci-dessous illustre les interactions entre les différents participants du patron de conception visiteur.



Application des patrons de conception : retour sur les fichiers

Dans cette section, nous reprenons la représentation des fichiers étudiée dans un précédent chapitre pour illustrer l'utilisation des différents patrons de conception présentés ci-dessus. Notez que les exemples donnés ici sont volontairement simples pour illustrer à la fois comment *choisir* le patron approprié, et comment l'*implémenter*. En pratique, les patrons de conception peuvent être utilisés pour réaliser des tâches bien plus complexes que celles présentées ici.

Un *proxy* pour limiter l'accès aux informations d'un fichier

On aimerait s'assurer qu'un utilisateur qui n'est pas le propriétaire d'un fichier ne puisse pas accéder aux informations relatives au contenu d'un fichier. En particulier, la taille ne doit pas être divulguée, et l'affichage d'un fichier doit simplement indiquer à qui appartient le fichier (et non pas afficher son contenu comme c'est le cas par défaut). En revanche, accéder au nom et à l'emplacement du fichier ainsi qu'au nom de son propriétaire ne pose pas de problème.

\pagebreak

Pour représenter cela, nous allons utiliser un *proxy* :

```
public class ProxyProprietaireFichier implements IFichier {

  private String utilisateur;

  private IFichier fichier;

  public ProxyProprietaireFichier(String utilisateur, IFichier fichier) {
    this.utilisateur = utilisateur;
    this.fichier = fichier;
  }

  @Override
  public String getNom() {
    return fichier.getNom();
  }

  @Override
  public String getEmplacement() {
    return fichier.getEmplacement();
  }

  @Override
  public int getTaille() {
    if (utilisateur.equals(fichier.getProprietaire())) {
      return fichier.getTaille();
```

```
    }
    throw new SecurityException("Accès refusé !");
  }

  @Override
  public String getProprietaire() {
    return fichier.getProprietaire();
  }

  @Override
  public String toString() {
    if (utilisateur.equals(fichier.getProprietaire())) {
      return fichier.toString();
    }
    return "Un fichier de " + fichier.getProprietaire();
  }

}
```

Pour utiliser ce proxy, il suffit de l'ajouter « autour » d'une instance de fichier, par exemple autour d'un fichier texte comme dans l'exemple ci-dessous :

```
IFichier fichier = new ProprietaireFichierProxy(nomUtilisateur,
    FichierRegulier.creerFichierRegulier(...));
```

Notez qu'il est possible d'ajouter un proxy sur un objet à n'importe quel moment, pas seulement tout de suite après sa création.

Une *stratégie* pour les algorithmes de chiffrement.

Pour assurer la sûreté des données contenues dans un fichier, nous souhaitons chiffrer celles-ci. Il existe de nombreuses manières de chiffrer les données, et on aimerait pouvoir choisir un algorithme de chiffrement ou un autre pour chiffrer nos fichiers. Ici, nous allons supposer que ce sont des chaînes de caractères que ces algorithmes prennent en paramètres, et que chaque algorithme doit pouvoir chiffrer ou déchiffrer des données (en fournissant la chaîne de caractères correspondante).

Pour cela, nous allons définir une stratégie, en définissant une interface pour les algorithmes de chiffrement que nous allons utiliser.

```
public interface AlgorithmeDeChiffrement {

  String chiffrer(String str);

  String dechiffrer(String str);

}
```

Cette interface doit ensuite être implémentée par les différents algorithmes possibles, par exemple le chiffrement de César donné ci-dessous :

```
public final class AlgorithmeDeCesar implements AlgorithmeDeChiffrement {

  private final int rotation;

  public AlgorithmeDeCesar(int rotation) {
    this.rotation = rotation;
  }

  @Override
  public String chiffrer(String str) {
    return appliquerRotation(str, rotation);
  }

  @Override
  public String dechiffrer(String str) {
    return appliquerRotation(str, 26 - rotation);
  }

  private String appliquerRotation(String str, int rot) {
    // ...
  }

}
```

Un *décorateur* pour chiffrer les fichiers

Maintenant que nous disposons de différents algorithmes de chiffrement, on souhaiterait pouvoir les appliquer pour chiffrer nos fichiers. En particulier, lorsque la méthode `toString()` est appelée, celle-ci doit retourner une version chiffrée de la chaîne de caractères décrivant le fichier lorsque celui-ci est chiffré. Nous souhaitons pouvoir appliquer le chiffrement à n'importe quel fichier existant. Pour ne pas avoir à modifier chacune des classes, nous allons plutôt définir un décorateur, qui permettra d'ajouter cette fonctionnalité à une instance de `IFichier` déjà existante.

```
public class FichierChiffreDecorateur implements IFichier {

  private AlgorithmeDeChiffrement chiffrement;

  private IFichier fichier;

  public FichierChiffreDecorateur(
      AlgorithmeDeChiffrement chiffrement, IFichier fichier) {
    this.chiffrement = chiffrement;
    this.fichier = fichier;
  }
```

```
  @Override
  public String getNom() {
    return fichier.getNom();
  }

  @Override
  public String getEmplacement() {
    return fichier.getEmplacement();
  }

  @Override
  public int getTaille() {
    return fichier.getTaille();
  }

  @Override
  public String getProprietaire() {
    return fichier.getProprietaire();
  }

  @Override
  public String toString() {
    return chiffrement.chiffrer(fichier.toString());
  }

}
```

Pour utiliser ce décorateur, il est par exemple possible de l'ajouter « autour » d'une instance de fichier, par exemple autour d'un fichier texte comme dans l'exemple ci-dessous :

```
IFichier fichier = new FichierChiffreDecorateur(
    new AlgorithmeDeCesar(13), FichierRegulier.creerFichierRegulier(...));
```

Notez qu'il est possible de décorer un objet à n'importe quel moment, pas seulement tout de suite après sa création.

Un adaptateur pour utiliser la classe `java.io.File` pré-existante

En *Java*, il existe déjà une représentation des fichiers, fournie par la classe `java.io.File`. Nous aimerions pouvoir utiliser des instances de cette classe dans notre système de gestion de fichiers, mais elle n'implémente pas l'interface `IFichier`. Afin de pouvoir exploiter cette implémentation, il est possible d'*adapter* cette classe :

```
public class FileAdaptateur implements IFichier {

  private File file;

  public FileAdaptateur(File file) {
    this.file = file;
  }

  @Override
  public String getNom() {
    return file.getName();
  }

  @Override
  public String getEmplacement() {
    return fichier.getPath();
  }

  @Override
  public int getTaille() {
    return (int) file.length();
  }

  @Override
  public String getProprietaire() {
    // Le code de cette méthode est omis par souci de simplicité.
  }

  @Override
  public String toString() {
    return file.toString();
  }

}
```

Pour utiliser cet adaptateur, il est par exemple possible de l'ajouter « autour » d'une instance de `File`, comme dans l'exemple ci-dessous :

```
IFichier fichier = new FileAdaptateur(new File(...));
```

Notez qu'il est possible d'adapter un objet à n'importe quel moment, pas seulement tout de suite après sa création.

Une *fabrique abstraite* pour créer les fichiers

Nous avons maintenant plusieurs types de fichiers possibles : des fichiers vérifiant l'identité de l'utilisateur, des fichiers chiffrés, des fichiers adaptés de la bibliothèque standard, *etc.* Nous souhaiterions pouvoir utiliser l'un ou l'autre de ces types de fichiers de manière transparente dans l'application, sans se soucier de leur type exact. Nous allons donc utiliser une fabrique abstraite, dont l'interface pourrait être la suivante :

```
public interface FabriqueDeFichiers {
```

```
  IFicher creerFichierRegulier(String nom, String emplacement,
      String proprietaire, byte[] contenu);

  IFicher creerFichierTexte(String nom, String emplacement,
      String proprietaire, String contenu);

}
```

Une première implémentation possible de cette interface consiste à simplement créer les fichiers tels quels, sans fonctionnalités supplémentaires :

```
public class FabriqueDeFichiersClassiques implements FabriqueDeFichiers {

  @Override
  public IFicher creerFichierRegulier(String nom, String emplacement,
      String proprietaire, byte[] contenu) {
    return FichierRegulier.creerFichierRegulier(
        nom, emplacement, proprietaire, contenu)
  }

  @Override
  public IFicher creerFichierTexte(String nom, String emplacement,
      String proprietaire, String contenu) {
    return FichierRegulier.creerFichierTexte(
        nom, emplacement, proprietaire, contenu)
  }

}
```

Si l'on souhaite plutôt créer des fichiers chiffrés, il faut définir une autre implémentation de l'interface, qui créera les décorateurs appropriés :

```
public class FabriqueDeFichiersChiffres implements FabriqueDeFichiers {

  @Override
  public IFicher creerFichierRegulier(String nom, String emplacement,
      String proprietaire, byte[] contenu) {
    return new FichierChiffreDecorateur(new AlgorithmeDeCesar(13),
        FichierRegulier.creerFichierRegulier(
            nom, emplacement, proprietaire, contenu)
  }

  @Override
  public IFicher creerFichierTexte(String nom, String emplacement,
      String proprietaire, String contenu) {
    return new FichierChiffreDecorateur(new AlgorithmeDeCesar(13),
        FichierRegulier.creerFichierTexte(
            nom, emplacement, proprietaire, contenu)
  }

}
```

On peut aussi définir une fabrique pour les fichiers adaptés de la bibliothèque standard *Java*. Notez que l'on n'utilise pas ici les informations relatives au nom, au propriétaire ou au contenu du fichier, comme la classe `File` est capable de retrouver seule ces informations à partir de l'emplacement du fichier.

```
public class FabriqueDeFichiersAdaptes implements FabriqueDeFichiers {

  @Override
  public IFicher creerFichierRegulier(String nom, String emplacement,
      String proprietaire, byte[] contenu) {
    return new FileAdaptateur(new File(emplacement));
  }

  @Override
  public IFicher creerFichierTexte(String nom, String emplacement,
      String proprietaire, String contenu) {
    return new FileAdaptateur(new File(emplacement));
  }

}
```

Pour créer un fichier, il suffit maintenant de passer par la fabrique, sans se soucier du type concret de l'objet créé par celle-ci :

```
FabriqueDeFichiers fabrique = ... ;
IFichier fichier = fabrique.creerFichierRegulier(...);
System.out.println(fichier.toString());
```

Un *état* pour verrouiller les fichiers

Lorsqu'un utilisateur accède à un fichier, on aimerait empêcher qu'un autre utilisateur puisse y accéder en même temps. En particulier, si le fichier est verrouillé, il ne doit pas être possible d'afficher son contenu (car il peut être en cours de modification). Pour cela, nous allons utiliser le patron de conception état, en définissant l'interface suivante :

```
public interface VerrouFichier {

  VerrouFichier changer();

  String getChaineFichier(IFichier fichier);
```

```
}
```

```
ر
```

D'un côté, la classe représentant l'état d'un fichier verrouillé en lecture pourrait être celle-ci :

```
public class FichierVerrouille implements VerrouFichier {

  @Override
  public VerrouFichier changer() {
    return new FichierLibre();
  }

  @Override
  public String getChaineFichier(IFichier fichier) {
    return "Fichier Verrouillé en lecture";
  }

}
```

D'un autre côté, la classe représentant l'état d'un fichier non verrouillé en lecture pourrait être celle-ci :

```
public class FichierLibre implements VerrouFichier {

  @Override
  public VerrouFichier changer() {
    return new FichierVerrouille();
  }

  @Override
  public String getChaineFichier(IFichier fichier) {
    return fichier.toString();
  }

}
```

On souhaiterait pouvoir ajouter le système de verrou à *n'importe quel fichier*. Pour cela, nous allons donc définir un décorateur, qui délèguera la méthode `toString()` au verrou pour s'assurer que le contenu du fichier ne puisse être affiché que lorsque le fichier n'est pas verrouillé :

```
public class FichierVerrouillable implements IFichier {

  private IFichier fichier;

  private VerrouFichier verrou;

  // ...

  @Override
  public String toString() {
    return verrou.getChaineFichier(fichier);
  }

}
```

Notez que l'on exploite ici le *polymorphisme* au lieu d'utiliser le conditionnel.

Un *singleton* pour des états uniques

Dans les états précédents, on voit que toutes les instances de `FichierLibre` et de `FichierVerrouille` sont identiques. Plutôt que de créer de nombreuses instances de ces états, il pourrait être intéressant de réutiliser toujours les mêmes, afin de limiter la création d'objets. Nous allons donc utiliser le patron singleton, dont une implémentation pour le `FichierLibre` est donnée ci-dessous :

```
public final class FichierLibre implements VerrouFichier {

  private static final FichierLibre INSTANCE = new FichierLibre();

  private FichierLibre() {
    // On ne fait rien ici.
    // Ce constructeur empêche l'instanciation.
  }

  public static VerrouFichier getInstance() {
    return INSTANCE;
  }

  @Override
  public VerrouFichier changer() {
    return FichierVerrouille.getInstance();
  }

}
```

Le singleton pour la classe `FichierVerrouille` est omis, comme son implémentation est très proche de celle du `FichierLibre`.

Le patron de méthode pour le stockage dans le *cloud*

On aimerait maintenant sauvegarder nos fichiers dans le *cloud*, en fournissant une implémentation pour différents fournisseurs. Pour sauvegarder un fichier dans le *cloud*, il faut se connecter au service, téléverser le fichier à l'emplacement souhaité puis se déconnecter du service. Néanmoins, ces différentes étapes ne sont pas implémentées de la même façon suivant le fournisseur. Le patron de méthode permet de s'assurer que les étapes sont toujours réalisées dans le même ordre, indépendamment de leur implémentation.

```
public abstract class SystemeDeSauvegarde {
```

```
  protected abstract void connexion();

  protected abstract void televersement(IFichier fichier);

  protected abstract void deconnexion();

  public final void sauvegarder(IFichier fichier) {
    connexion();
    televersement(fichier);
    deconnexion();
  }

}
```

Ici, c'est la méthode `sauvegarder()` qui implémente le patron de méthode. Elle est déclarée `final` pour s'assurer qu'elle ne puisse pas être redéfinie (mais il peut parfois être utile de l'autoriser malgré tout). Les autres méthodes sont déclarées `protected` car elles ne doivent pas être appelées seules, en dehors de la méthode `sauvegarder`. Elles sont ici abstraites, mais on pourrait aussi définir une implémentation de base pour chacune de ces méthodes.

Pour spécifer le comportement des différentes étapes, il faut étendre la classe `SystemeDeSauvegarde`, *a priori* pour chacun des fournisseurs de service de *cloud* que l'on souhaite considérer, par exemple :

```
public class OwnCloud extends SystemeDeSauvegarde {

  @Override
  protected void connexion() {
    // ...
  }

  @Override
  protected void televersement(IFichier fichier) {
    // ...
  }

  @Override
  protected void deconnexion() {
    // ...
  }

}
```

Le *composite* pour les répertoires

Nous voudrions maintenant définir des répertoires pour notre système de gestion de fichiers. Comme sur les systèmes UNIX, un répertoire doit pouvoir être utilisé comme n'importe quel autre fichier. Cependant, il est lui même constitué de plusieurs autres fichiers (qui peuvent aussi être des répertoires). Nous allons donc appliquer le patron composite pour représenter un répertoire :

```
public class Repertoire extends AbstractFichier {

  private String emplacement;

  private Collection<IFichier> fichiers;

  public Repertoire(String nom, String proprietaire,
      String emplacement) {
    super(nom, proprietaire);
    this.files = new LinkedList<>();
  }

  // ...

  @Override
  public int getTaille() {
    int taille = 0;
    for (IFichier f : fichiers) {
      taille += f.getTaille();
    }
    return taille;
  }

  @Override
  public String toString() {
    StringBuilder sb = new StringBuilder("Répertoire ");
    sb.append(getNom()).append("\n");
    for (IFichier f : fichiers) {
      sb.append(f).append("\n");
    }
    return sb.toString();
  }

}
```

Notez ici que les méthodes du répertoire permettent d'aggréger les résultats fournis par ces mêmes méthodes appelées sur les fichiers contenus dans ce répertoire.

L'*itérateur* pour parcourir les fichier d'un répertoire

La classe `Repertoire` définit un attribut permettant de stocker les fichiers contenus dans le répertoire. Pour connaître les différents fichiers qu'un répertoire contient, on aimerait pouvoir les parcourir (par exemple, pour les afficher, comme le ferait `ls`). Pour cela, nous allons utiliser le patron itérateur, en implémentant l'interface `Iterable<T>` définie dans la bibliothèque standard *Java*.

```
public class Repertoire extends AbstractFichier implements Iterable<IFichier> {

  // ...

  @Override
  public Iterator<IFichier> iterator() {
    return fichiers.iterator();
  }

}
```

Ici, on se contente d'exploiter le fait qu'une `Collection` implémente `Iterable`, mais il faut parfois définir sa propre implémentation d'`Iterator`.

L'implémentation du patron itérateur dans la classe `Repertoire` nous permet maintenant de parcourir les fichiers qu'il contient à l'aide d'un `for-each` :

```
for (IFichier fichier : monRepertoire) {
  // Faire quelque chose avec le fichier courant...
}
```

Le *visiteur* pour explorer la hiérarchie des fichiers

Lister le contenu d'un répertoire n'est pas toujours suffisant. On a parfois besoin d'explorer l'intégralité de la hiérarchie, par exemple pour connaître tous les fichiers qu'elle contient, ainsi que leur type. C'est la patron visiteur qui permet cela.

Dans un premier temps, il faut définir une interface pour le visiteur, qui déclare une méthode `visit()` **pour chaque type concret** implémentant l'interface `IFichier` :

```
public interface FichierVisiteur {

  void visit(Repertoire repertoire);

  void visit(FichierRegulier fichier);

  void visit(FichierTexte fichier);

  void visit(ProxyFichier fichier);

  void visit(FichierChiffreDecorateur fichier);

  void visit(FileAdaptateur fichier);

}
```

Une implémentation possible du visiteur pourrait être d'afficher le nom de chacun des fichiers parcourus, avec un préfixe dépendant de son type :

```
public class Listeur implements FichierVisiteur {

  @Override
  public void visit(Repertoire repertoire) {
      System.out.println("d " + repertoire.getNom());
  }

  @Override
  public void visit(FichierRegulier fichier) {
      System.out.println("f " + fichier.getNom());
  }

  @Override
  public void visit(FichierTexte fichier) {
      System.out.println("t " + fichier.getNom());
  }

  // ...

}
```

Pour implémenter d'autres opérations, il suffit de fournir d'autres implémentations de l'interface du visiteur. Pour pouvoir les réaliser, il faut fournir le visiteur aux fichiers, en ajoutant à l'interface `IFichier` pour permettre d'*accepter* un visiteur :

```
public interface IFichier {

  // ...

  void accept(FichierVisiteur visiteur);

}
```

Cette méthode doit permettre d'explorer toute la hiérarchie, en partant de la racine. Dans le cas du répertoire, son implémentation pourrait être la suivante :

```
public class Repertoire extends AbstractFichier {

  // ...
```

```
  @Override
  public void accept(FichierVisiteur visiteur) {
    visiteur.visit(this);
    for (IFichier f : fichiers) {
      f.accept(visiteur);
    }
  }

}
```

Notez que l'on ***visite l'objet courant***, et que l'on fait **accepter le visiteur** aux objets « enfants », afin de laisser à chaque fichier la responsabilité du parcours : c'est le principe du **double dispatch**. Pour utiliser le visiteur, il faut de la même manière le faire accepter par le fichier que l'on souhaite visiter :

```
Repertoire repertoire = new Repertoire();
// ...
FichierVisiteur visiteur = new Listeur();
repertoire.accept(visiteur);
```